DEEP NEURAL NETWORKS (AIML ZG565) · COMPANION READER

Introduction to Deep Learning

What deep learning is, why it works now, and the four ingredients of every deep learning problem

Session 1 (26 April 2026) · Read alongside the lecture slides · Every slide example worked out in full

How to use this companion. The slides give you the formulas; this reader gives you the *why*. Every slide concept is expanded into a full, plain-English explanation, and every slide example is worked out in full, step by step. Colour-coded boxes guide you: the **blue** "intuition" box states the core idea in one breath; the **amber** "everyday picture" box ties it to real life before any math; the **green** "worked example" box solves a problem with real numbers and no skipped steps; the **red** "watch out" box flags the common mistake; and the **purple** "key takeaway" box is the one sentence to remember. Read it alongside the slides, in order.

1 What this session covers

Session 1 is the front door of the course. It answers four questions, in order. First: *what* is deep learning, and where does it sit inside AI and machine learning? Second: *why* did it take off now, after decades of quiet? Third: what are the *four core ingredients* of every deep learning problem — data, a model, an objective function, and a learning algorithm? Fourth: what *kinds* of learning problems exist — supervised, unsupervised, and reinforcement learning — and which tasks live under each?

For course logistics: the textbook is *Dive into Deep Learning* (d2l.ai) and the reference is *Deep Learning* by Goodfellow, Bengio, and Courville. Grading is Quiz I and II (5% each), two assignments (10% each), a mid-semester exam (30%, Sessions 1–8), and a comprehensive exam (40%, Sessions 1–16). Materials live on Taxila; recordings on Microsoft Teams. Suggested reading for this session: Chapter 1 of the textbook.

2 What is deep learning?

Let us start with the words themselves. The slides give five definitions of **deep learning**. They all say the same thing from different angles: deep learning is a kind of **machine learning** (learning from examples) that uses an **artificial neural network** (a stack of simple computing layers) with *several* layers, where each layer extracts a slightly more meaningful description of the data than the layer before it.

☞ The intuition

Deep learning = machine learning with many *stacked layers*. Each layer takes the previous layer's output and re-describes it in a slightly more useful way. "Deep" is not a claim about deep understanding — it literally counts the layers. The number of layers is called the **depth** of the model.

★ Everyday picture

Think of cleaning a noisy photo of a friend's face in stages. Stage one finds edges. Stage two groups edges into shapes: an oval here, two circles there. Stage three names parts: eyes, nose, mouth. Stage four says "that's Priya." No single stage is clever. The *stack* is clever. Deep learning builds exactly this kind of assembly line — and, crucially, it *learns* what each stage should look for, instead of being told.

These layered re-descriptions are called **representations**. A representation is just a way of writing the data as numbers — raw pixels are one representation; "edges present at these positions" is another, more useful one. Deep learning is therefore often described as *learning successive layers of increasingly meaningful representations*. The layered models that do this are called **neural networks**: literal stacks of layers, one on top of the other.

The 'deep' in deep learning: successive layers of representation

Each layer turns the previous layer's output into a slightly more meaningful description. The number of layers is the model's depth.

Figure 1: How to read it: follow the boxes left to right. Raw pixels become edges, edges become textures and corners, parts emerge, and finally a decision. Each arrow is one learned layer.

Where this shows up in ML. This is the course in one picture. A convolutional network (Session 6 onward) really does learn edges in early layers and object parts in later ones. A transformer (later sessions) does the same with words: early layers see spelling-level patterns, deep layers see meaning. Depth is the common thread.

✗ Watch out

"Deep" does not mean "profound" or "conscious." It is a count of layers — three or more is commonly called deep. Also, deep learning is not separate from machine learning; it is a subfield *inside* it.

✓ Key takeaway

Deep learning is machine learning with many stacked layers, each learning a more meaningful representation of the data than the last.

3 Where deep learning sits: AI ⊃ ML ⊃ DL

The three famous letters fit inside each other like Russian dolls. **Artificial intelligence (AI)** is the broadest goal: the science of making machines perform human tasks — for example, a robot cleaning a room. AI includes many approaches that involve no learning at all, such as hard-coded rules. **Machine learning (ML)** is one approach to AI: instead of writing the rules by hand, the machine *learns* them from experience (data). **Deep learning (DL)** is one technique for doing machine learning: it recognises

patterns using deep neural networks.

☞ The intuition

AI is the goal (act smart). ML is a strategy for the goal (learn from data rather than follow fixed rules). DL is a tool for the strategy (learn with many-layered networks). Every DL system is an ML system, and every ML system is an AI system — but not the other way round.

★ Everyday picture

Think of "transport ⊃ cars ⊃ electric cars." Transport is the broad goal of moving people (AI: act smart). Cars are one way to do transport (ML: learn from data). Electric cars are one kind of car (DL: learn with deep networks). Saying "AI and deep learning" as if they were rivals is like saying "transport and electric cars" — one contains the other.

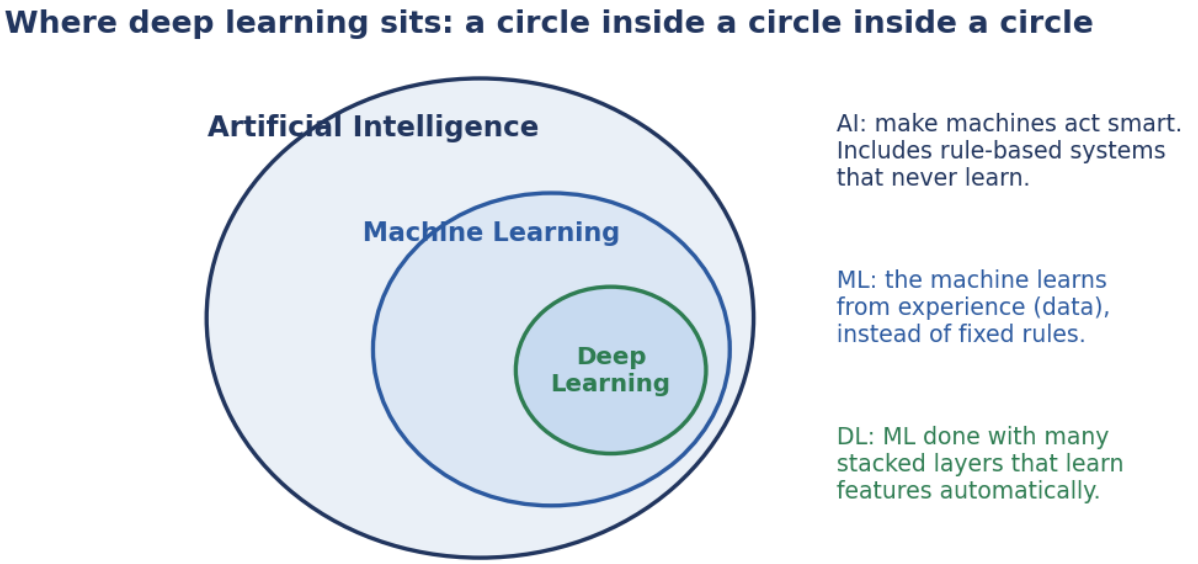

Figure 2: How to read it: each circle sits fully inside the bigger one. A rule-based chess program is in the AI circle but outside ML; a spam filter trained on examples is in ML; an image recogniser with 50 layers is in the innermost DL circle.

Where this shows up in ML. Interview questions and exam questions love this distinction. More practically, it tells you when *not* to use deep learning: if a task has clear fixed rules (compute tax from a formula), plain AI programming wins; if data is tiny and features are simple, classical ML (say, a decision tree) often beats a deep network.

✗ Watch out

Do not say "AI versus machine learning" — ML is inside AI. And not all AI learns: a chess engine following hand-written rules is AI with zero learning.

✓ Key takeaway

AI is the goal, ML is learning from data to reach it, and DL is ML built from many-layered neural networks: three nested circles, not three rivals.

4 Why deep learning, why now?

Neural networks are old — the perceptron dates to the 1950s. So why did deep learning explode only in the last fifteen years? The slides give a three-ingredient answer: **data**, **computation**, and **algorithms**, and a fourth practical one, **scalability**.

- **Data and storage.** The internet, phones, and cheap sensors produce enormous amounts of data — much of it **unstructured** data like images, text, audio, and video (data with no neat table of rows and columns). Storage to keep it became cheap.
- **Computation.** CPUs, and especially **GPUs** (graphics processors that do thousands of small multiplications in parallel) and distributed clusters, made training big models affordable.
- **Algorithms.** Deep models learn by example and generate their own features automatically (this is called **automated feature generation**), instead of needing experts to hand-design them. Learning capability improved.
- **Scalability.** The same recipe keeps working as you add more data and more compute, so advanced analytics can be applied at scale.

★ Everyday picture

Why did food delivery apps appear around 2014 and not 1994? Not because the idea was new — because the ingredients finally co-existed: smartphones in every pocket (data), fast mobile networks (computation), and good maps and routing (algorithms). Deep learning had the same story: the idea waited decades for its ingredients.

The key empirical fact is in the figure below: with small data, classical methods and deep networks perform similarly (classical methods often win). As data grows, hand-made features **saturate** — they stop improving — while deeper networks keep climbing.

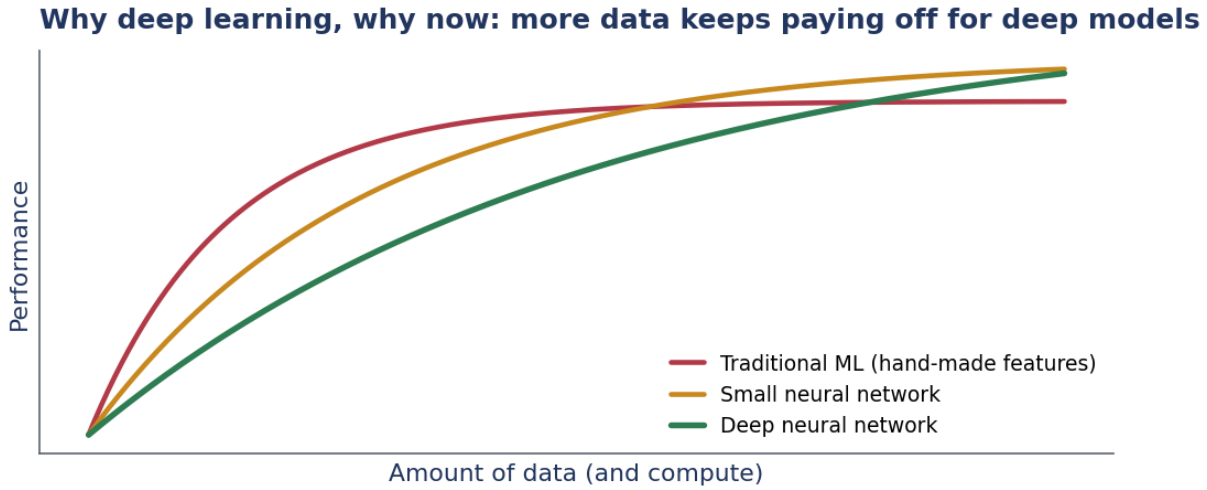

Figure 3: How to read it: move along the horizontal axis (more data). The red curve flattens early; the green curve keeps rising. The gap on the right side is the modern deep learning era.

Where this shows up in ML. This curve drives real engineering decisions. With 500 labelled rows, choose a classical model. With 5 million images, a deep network will likely dominate. It also explains the industry race for data and GPUs: both axes of this plot cost money.

✗ Watch out

"Deep learning is always better" is false. On small, structured, tabular datasets, gradient-boosted trees frequently beat deep networks. The advantage of depth appears with scale.

✓ Key takeaway

Deep learning works now because big data, cheap parallel compute, and better algorithms finally arrived together — and deep models keep improving with scale where hand-made features stall.

5 Applications and breakthroughs

Where has this paid off? The slides list breakthrough after breakthrough: image recognition at human level, speech recognition in every phone, machine translation, game-playing systems, and generative models. They also list everyday applications worth reading carefully, because each one quietly names a *task type* you will meet later in the course:

- Predict tomorrow's weather from geographic information, satellite images, and a trailing window of past weather — a **regression** problem on sequences and images.
- Take a question in free-form text and answer it correctly — **question answering (Q&A)**, the seed of today's chat assistants.
- Take an image and outline every person in it — **object detection**.
- Present users with products they would enjoy but would not naturally stumble upon — **recommender systems**.

☞ The intuition

Every flashy application is one of a few repeating task shapes: predict a number, predict a category, find things in an image, rank items, or transform one sequence into another. Learn the shapes once, and the application list stops being a zoo.

Where this shows up in ML. Each bullet maps to a later session: convolutional networks for images (object detection), recurrent networks and attention for sequences (weather windows, translation), transformers for Q&A, and embedding models for recommendation.

✓ Key takeaway

Deep learning's breakthroughs — vision, speech, translation, games, recommendation — are a small set of task shapes wearing different clothes.

6 Ingredient 1 of 4: Data

Every deep learning problem is built from the same four core components: the *data* we learn from, a *model* that transforms data, an *objective function* that scores the model, and an *algorithm* that improves the model. We take them one at a time.

Data is a collection of examples. Each **example** (also called a data point, data instance, or sample) is a set of measured attributes called **features** (or covariates) — the inputs the model predicts from. In **supervised learning**, one special attribute is the thing to be predicted; it is called the **label** (or target). Whatever the raw data is — pixels, words, audio — it must first be converted into a useful *numerical*

representation, because models compute with numbers only.

The slide's notation: the dataset is $D = \{X, t\}$, where X holds the features of all examples and t holds their labels. The number of features per example is d, also called the number of **dimensions**; the number of examples is N. As the number of features grows, we say the **dimensionality** increases.

★ Everyday picture

A class register is a dataset. Each row is one student (an example, N rows in total). The columns — attendance, quiz 1 score, quiz 2 score — are features (d columns). The final-grade column is the label t. "Higher dimensionality" just means someone added more columns.

✎ Worked example: counting N and d in a tiny house-price dataset

We have data on three houses. For each house we record its size in square feet, the number of bedrooms, and its age in years. We want to predict price.

Step 1. Write the raw table. House A: (1,000 sq ft, 2 bedrooms, 10 years) → price 50 lakh. House B: (1,500, 3, 5) → 75 lakh. House C: (2,000, 4, 2) → 98 lakh.

Step 2. Identify the features. The inputs are size, bedrooms, and age. Count them: $d = \mathbf{3}$ features, so each house is a point in 3-dimensional space.

Step 3. Identify the label. The attribute to predict is price, so $t = (50, 75, 98)$ holds the labels.

Step 4. Count the examples. There are three houses, so $N = \mathbf{3}$.

Step 5. Assemble $D = \{X, t\}$. Stack the feature rows into the matrix

$$X = \begin{bmatrix} 1000 & 2 & 10 \\ 1500 & 3 & 5 \\ 2000 & 4 & 2 \end{bmatrix}, \qquad t = \begin{bmatrix} 50 \\ 75 \\ 98 \end{bmatrix}. \tag{1}$$

X has shape $N \times d = 3 \times 3$.

Step 6. Sense-check. Rows = houses (N), columns = measurements (d), and t has one entry per row. Consistent.

The whole of deep learning starts by putting your problem into exactly this $\{X, t\}$ shape.

Where this shows up in ML. Every framework call you will ever write — `model.fit(X, t)` — expects this exact shape. Images become X by flattening or stacking pixels; text becomes X via token numbers. The "curse of dimensionality" (problems that appear when d gets huge) motivates representation learning itself.

✗ Watch out

Do not confuse N and d. N is how many examples you have (rows); d is how many numbers describe each example (columns). "Big data" usually means big N; "high-dimensional data" means big d.

✓ Key takeaway

Data is $D = \{X, t\}$: N examples, each described by d features, with labels t when the task is supervised — always converted to numbers first.

7 Ingredient 2 of 4: The model

A **model** is the computational machinery that ingests data of one type and spits out predictions of a possibly different type — pixels in, "cat" out; audio in, yes/no out. A deep learning model is many successive transformations of the data chained together top to bottom; that chaining is literally why the field is called *deep* learning.

☞ The intuition

A model is a function with adjustable knobs. The input goes in, a prediction comes out, and the knobs — called **parameters** — decide exactly how the input is transformed. Learning means turning the knobs well.

★ Everyday picture

A kitchen mixer-grinder is a model. Ingredients go in (input), a paste comes out (prediction), and the speed dial and timer are the parameters. The same machine makes chutney or batter depending on how you set the dials. Deep models just have millions of dials and set them automatically.

✎ Worked example: the smallest possible model making a prediction

Take the simplest model family: predicted price $= w \times \text{size} + b$, where w (a weight) and b (a bias) are the two parameters. Suppose learning has set $w = 0.05$ and $b = 1$, with size in square feet and price in lakh. Predict the price of House A (1,000 sq ft).

Step 1. Write what we know. Input size $x = 1000$; parameters $w = 0.05$, $b = 1$.

Step 2. Plug into the model.

$$\hat{y} = wx + b = 0.05 \times 1000 + 1. \tag{2}$$

The hat on $\hat{y}$ marks a *prediction*, to distinguish it from the true label y.

Step 3. Multiply. $0.05 \times 1000 = \mathbf{50}$.

Step 4. Add the bias. $50 + 1 = \mathbf{51}$. So the model predicts **51 lakh**.

Step 5. Sense-check. The true price was 50 lakh, so the model is off by 1 lakh — close, not perfect. Quantifying that gap is exactly the job of the next ingredient.

A deep network does this same plug-in arithmetic, just through many layers of weights instead of one.

Where this shows up in ML. Every architecture in this course — the perceptron (Session 2), feedforward networks, CNNs, transformers — is a model in precisely this sense: a parameterised function from inputs to predictions. GPT-style models have billions of these w-like parameters.

✗ Watch out

A model is not the same as an algorithm. The model is the machine with knobs; the learning algorithm (ingredient 4) is the procedure that turns the knobs.

✓ Key takeaway

A model is a parameterised transformation from inputs to predictions; deep models chain many such transformations, layer after layer.

8 Ingredient 3 of 4: The objective (loss) function

Learning means *improving* at a task over time — but improving needs a score. An **objective function** is a formal measure of how good (or bad) a model currently is. By convention objective functions are defined so that *lower is better*; because of that convention they are often called **loss functions**.

☞ The intuition

The loss is the model's exam mark, flipped: zero is a perfect score, and bigger numbers mean worse predictions. Training is simply the project of driving this one number down.

★ Everyday picture

Think of golf. Every hole gives a score, and *lower is better*. A pro and a beginner can both play the same course; the scorecard is what tells them apart. The loss function is the model's scorecard — one number that lets us compare any two settings of the knobs.

A standard loss for predicting numbers is the **squared error**: for one example, take the gap between the true label y and the prediction $\hat{y}$, and square it, $(y-\hat{y})^2$. Squaring does two jobs: it makes undershoot and overshoot both count as positive error, and it punishes big misses much more than small ones. Averaging over all N examples gives the **mean squared error (MSE)**:

$$L = \frac{1}{N}\sum_{i=1}^{N}(y_i-\hat{y}_i)^2, \tag{3}$$

where y_i is the true label of example i and $\hat{y}_i$ is the model's prediction for it.

✎ Worked example: scoring our house model with mean squared error

Use the model $\hat{y} = 0.05x + 1$ on all three houses and compute its MSE.

Step 1. Predict each house. House A: $0.05 \times 1000 + 1 = \mathbf{51}$. House B: $0.05 \times 1500 + 1 = \mathbf{76}$. House C: $0.05 \times 2000 + 1 = \mathbf{101}$.

Step 2. Find each gap (true − predicted). A: $50 - 51 = \mathbf{-1}$. B: $75 - 76 = \mathbf{-1}$. C: $98 - 101 = \mathbf{-3}$.

Step 3. Square each gap. $(-1)^2 = \mathbf{1}$; $(-1)^2 = \mathbf{1}$; $(-3)^2 = \mathbf{9}$.

Step 4. Average over $N = 3$.

$$L = \frac{1+1+9}{3} = \frac{11}{3} = \mathbf{3.6667}. \tag{4}$$

Step 5. Sense-check. The loss is positive (squares always are), and it is dominated by House C's miss of 3 — squaring made that one mistake count 9, nine times a miss of 1. A different (w, b) giving a smaller L would be a better model; the scorecard now lets us compare.

So this model scores **3.6667**; training will hunt for parameters that score lower.

Where this shows up in ML. Every training run you will ever launch prints a falling "loss" curve — this number. Regression uses MSE; classification uses cross-entropy (a loss over probabilities you will meet in Session 4); language models minimise cross-entropy over next words.

✗ Watch out

Lower is better — always check the convention. A loss of 0.01 is not "1% accuracy"; loss and accuracy are different scales, and a falling loss is good while falling accuracy is bad.

✓ Key takeaway

The objective (loss) function turns "how good is the model?" into a single number where lower is better — giving training something concrete to minimise.

9 Ingredient 4 of 4: The learning algorithm — gradient descent

The final ingredient is the **learning algorithm**: a procedure that searches for the best possible parameters — the ones that minimise the loss. The workhorse of deep learning is **gradient descent**.

☞ The intuition

At your current parameter setting, ask: "if I nudge this knob slightly, does the loss go up or down, and how fast?" That local slope is the **gradient**. Gradient descent repeats two moves: read the slope, then step *against* it (downhill). Loop until the ground is flat.

★ Everyday picture

You are on a hillside in thick fog and want the valley. You cannot see the valley — but you can *feel* the slope under your boots. So you take a step downhill, feel again, step again. The slope under your boots is the gradient; the size of each stride is the **learning rate**; the valley is the minimum loss. (Where the analogy breaks: a real walker has momentum and memory; plain gradient descent has neither — fixes for that come in the optimisation session.)

Formally, write the loss as a function of a parameter, $L(w)$. The update rule is

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{dL}{dw}, \tag{5}$$

read symbol by symbol: w_{old} is the current knob setting; $\frac{dL}{dw}$ is the gradient — the slope of the loss at that setting (positive slope means "loss rises if w rises"); η, the Greek letter *eta*, is the **learning rate**, a small positive number controlling the step size; and the minus sign points the step *downhill*.

✎ Worked example: two steps of gradient descent, by hand

Take a toy loss $L(w) = (w-3)^2 + 1$, which is a bowl with its bottom at $w = 3$. Its slope is $\frac{dL}{dw} = 2(w-3)$. Start at $w_0 = 0$ with learning rate $\eta = 0.3$ and take two steps.

Step 1. Write what we know. $w_0 = 0$, $\eta = 0.3$, gradient formula $2(w-3)$.

Step 2. Slope at $w_0 = 0$. $2(0-3) = \mathbf{-6}$. Negative slope: the loss falls as w rises, so we should move right.

Step 3. First update.

$$w_1 = w_0 - \eta \times (-6) = 0 + 0.3 \times 6 = \mathbf{1.8}. \tag{6}$$

Step 4. Check the loss dropped. $L(w_0) = (0-3)^2 + 1 = \mathbf{10}$; $L(w_1) = (1.8-3)^2 + 1 = 1.44 + 1 = \mathbf{2.44}$. Down from 10 to 2.44. Good.

Step 5. Slope at $w_1 = 1.8$. $2(1.8-3) = \mathbf{-2.4}$ — still negative, but smaller: the ground is flattening near the valley.

Step 6. Second update.

$$w_2 = 1.8 - 0.3 \times (-2.4) = 1.8 + 0.72 = \mathbf{2.52}. \tag{7}$$

Loss check: $L(w_2) = (2.52-3)^2 + 1 = 0.2304 + 1 = \mathbf{1.2304}$.

Step 7. Sense-check. The iterates $0 \to 1.8 \to 2.52$ march toward the true minimum $w = 3$, and each step is smaller than the last because the slope shrinks near the bottom. Exactly the behaviour we wanted.

The trick to remember: the step size is $\eta \times$ slope, so steps shrink automatically as the valley flattens.

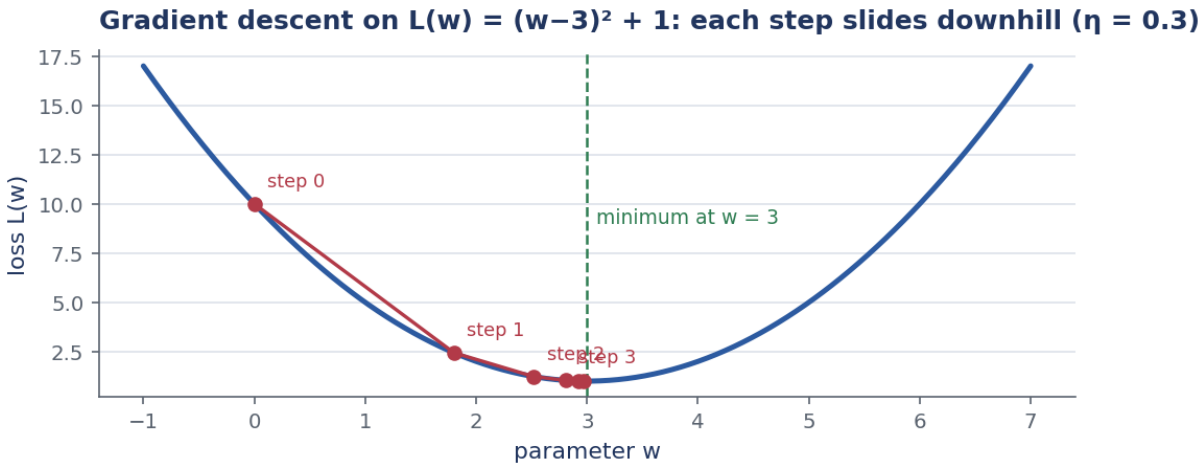

Figure 4: How to read it: the blue bowl is the loss $L(w)$; the red dots replay the worked example's iterates from $w_0 = 0$, sliding downhill toward the dashed minimum at $w = 3$.

Where this shows up in ML. Gradient descent (and its variants SGD and Adam) trains essentially every deep network in existence. **Backpropagation** — coming in a later session — is nothing more than an efficient recipe for computing these gradients through many layers at once.

✗ Watch out

The learning rate η is the most fragile dial in deep learning. Too small and training crawls; too large and each step overshoots the valley, so the loss bounces or even *grows*. Re-run the worked example with $\eta = 1.1$ and watch w fly past 3 and oscillate outward.

✓ Key takeaway

Gradient descent improves a model by repeatedly stepping each parameter against its loss slope, $w \leftarrow w - \eta\, dL/dw$, with the learning rate η setting the stride.

10 The framework end to end: the wake-word example

The slides walk the four ingredients through one concrete problem: build a system that wakes a phone when it hears a phrase like "Alexa" or "Hey Siri." This is the **wake-word** problem, and it shows *why* we need learning at all.

Could we just program it directly? We would have to tell the computer explicitly how to map inputs to outputs — to write rules over raw audio samples that fire on "Alexa" in any voice, accent, speed, and background noise. Nobody knows how to write those rules. What we *can* do is define the problem precisely (audio snippet in; yes/no out), choose an appropriate **model family**, and collect a huge dataset of audio clips labelled "contains the wake word" or "does not."

☞ The intuition

When the rules are too subtle for humans to write down, flip the workflow: instead of programming the behaviour, program a *parameterised* machine and let data choose the parameters. The program whose parameters are still undecided becomes a *model* once data has decided them.

★ Everyday picture

Teaching a child to recognise grandma's voice on the phone: you never give rules ("her voice has these frequencies"). You let the child hear many calls — "that's grandma," "that's not" — and the recognition forms by example. Wake-word training is the same: thousands of labelled clips, no hand-written rules.

✎ Worked example: the wake-word problem, mapped onto the four ingredients

We assemble the slide's example into the full framework, piece by piece.

Step 1. Define inputs and outputs precisely. Input: a one-second audio snippet, converted to numbers (say 16,000 samples per second, so $d = 16{,}000$ features). Output: one of {yes, no}.

Step 2. Data. Collect a huge labelled set: clips containing the wake word (label = yes) and clips not containing it (label = no). If we gather one million clips, $N = 10^6$.

Step 3. Model. Start with a program whose behaviour is determined by a number of parameters — a network mapping 16,000 numbers to a yes/no score. Before training, its parameters are arbitrary and it answers randomly.

Step 4. Objective. A loss that is low when the model says "yes" on wake-word clips and "no" on the rest, and high otherwise.

Step 5. Learning algorithm. Run gradient descent: show clips, score them with the loss, nudge every parameter downhill, repeat. The parameters improve the program's performance with respect to the loss.

Step 6. Freeze and name. Once the parameters are finalised, we call the program a **model**: snippet in, yes/no out.

Step 7. Sense-check. Did we ever write a rule about accents or noise? No — robustness was forced by variety in the data, which is exactly the point of the example.

One more idea from the slides: the set of all distinct programs (input–output mappings) we can produce just by changing the parameters is called a **family of models**. The same family should suit "Alexa" recognition and "Hey Siri" recognition, because the tasks are intuitively similar — only the learned parameters differ.

Where this shows up in ML. This is transfer thinking in embryo: one architecture, many tasks. A ResNet family serves cats-vs-dogs and tumour detection alike; a transformer family serves translation and chat. Choosing the family is engineering; choosing the parameters is training.

✗ Watch out

"Model" and "model family" are different. The family is the menu of all programs reachable by turning the knobs; the model is the single dish you end up with after training picks specific knob values.

✓ Key takeaway

When rules are unwritable, define the input–output contract, pick a model family, and let data plus gradient descent pick the parameters — the trained result is the model.

11 Kinds of learning problems

Deep learning problems come in three broad kinds, distinguished by what the data tells the learner.

Supervised learning is training by examples: the model receives a dataset with both the data *and* its labels. Classification and regression are the canonical cases. This is the kind you will study in this course (and in the Machine Learning course).

Unsupervised learning covers situations where we feed the model a giant dataset containing *only the features* — no labels. The model must find structure on its own: grouping similar items (**clustering**), or learning to generate realistic synthetic data (for example with **GANs**, generative adversarial networks). Covered in the Advanced Deep Learning course.

Reinforcement learning (RL) develops an **agent** that interacts with an **environment** and takes actions over a series of time steps, learning from rewards rather than labelled answers. Covered in the Deep Reinforcement Learning course.

★ Everyday picture

Three ways to learn cooking. Supervised: a teacher stands beside you and corrects every dish ("this one is too salty") — labelled feedback per example. Unsupervised: you are locked in a kitchen with ingredients and no teacher; you discover for yourself which ingredients group well. Reinforcement: you run a food stall; nobody labels your dishes, but daily sales (rewards) slowly teach you which recipes work.

Where this shows up in ML. Modern systems mix all three: a large language model is pre-trained on next-word prediction (self-supervised, a label-free cousin of supervised learning), then refined with reinforcement learning from human feedback.

✗ Watch out

The split is about the *data*, not the model. The same network architecture can be trained supervised one day and reused inside an RL agent the next. Ask "does each example carry a label?" — that decides the kind.

✓ Key takeaway

Supervised learns from labelled examples, unsupervised finds structure in unlabelled features, and reinforcement learns from rewards earned by acting.

12 Supervised learning, up close

Since this course lives mostly in supervised territory, the slides zoom in. Each feature–label pair is called an **example**. The goal is a model that maps *any* input to a label prediction — not just the inputs we trained on. The "supervision" is us: we, the supervisors, choose the parameters by providing a dataset of labelled examples, where each example links input features to (in probability language) the **conditional probability of a label given the input features**, written $P(t \mid x)$ — "the probability of label t once you know the features x."

The slides then catalogue the main supervised *task types*:

- **Regression** — answers "how much / how many?" with a number (house price, tomorrow's temperature).
- **Classification** — answers "which one?" with a category. It comes in three flavours: **binary** (two classes: spam / not spam), **multi-class** (one label from many: which digit, 0–9), and **multi-label** (several labels at once: a news article can be both "politics" and "economy").
- **Recommender systems** — predict which items a particular user will like.
- **Sequence learning** — inputs or outputs are ordered sequences: **tagging and parsing** (label every word in a sentence), **automatic speech recognition** (audio → text), **text to speech** (text → audio), and **machine translation** (sentence in one language → sentence in another).

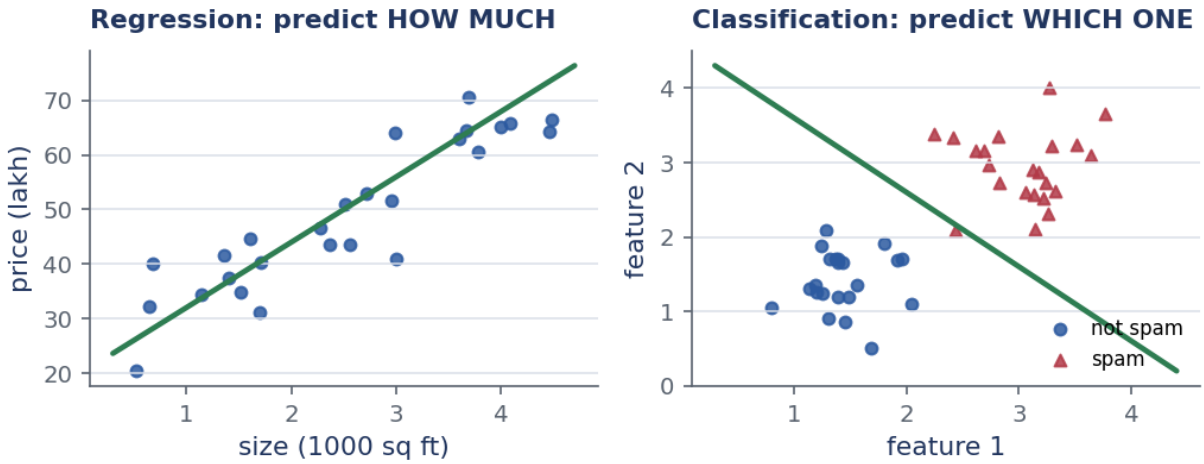

Figure 5: How to read it: left panel — regression fits a line through numeric labels (price vs size). Right panel — classification draws a fence between two labelled groups. Same supervised recipe, different output types.

✎ Worked example: naming the task type, four quick calls

For each scenario, identify the supervised task type and say why.

Step 1. **Predict a house's selling price from its features.** The answer is a number on a continuous scale ⇒ **regression**.

Step 2. **Decide whether an email is spam.** Exactly two possible labels ⇒ **binary classification**.

Step 3. **Read a handwritten digit.** One label from the ten classes 0–9 ⇒ **multi-class classification**.

Step 4. **Tag a movie with all genres that apply** (a film can be both comedy and romance). Several labels may be true simultaneously ⇒ **multi-label classification**.

The deciding question is always: *what shape is the answer?* A number → regression; one class → (binary or multi-class) classification; many classes at once → multi-label.

Where this shows up in ML. The task type fixes your last layer and your loss: regression ends in a plain linear output with squared-error loss; multi-class ends in a softmax with cross-entropy; multi-label ends in independent sigmoids. Sessions 3–5 build exactly these heads.

✗ Watch out

Multi-class and multi-label are easy to confuse. Multi-class: the classes compete, exactly one wins (a digit is a 7, full stop). Multi-label: each label is its own yes/no question, and several can be yes.

✓ Key takeaway

Supervised learning maps features to label predictions; the shape of the answer — number, one class, or many classes — names the task and picks the loss.

13 Recap and what comes next

One paragraph to carry away. Deep learning is machine learning with stacked, learned layers of representation, sitting inside ML, which sits inside AI. It took off now because data, compute, and algorithms finally arrived together. Every deep learning problem has four ingredients: *data* ($D = \{X, t\}$, N examples, d features), a *model* (a parameterised transformation), an *objective* (a loss where lower is better), and a *learning algorithm* (gradient descent: $w \leftarrow w - \eta\, dL/dw$). Problems come in three kinds — supervised, unsupervised, reinforcement — and supervised tasks split by the shape of the answer: regression, classification (binary, multi-class, multi-label), recommendation, and sequence learning.

✓ Key takeaway

Data + model + loss + learning algorithm: spot these four in any deep learning system and you have understood its skeleton. Session 2 zooms into the model box, starting with the artificial neuron and the perceptron.

Further reading. Dive into Deep Learning (textbook T1), Chapter 1 — it covers everything in this session with runnable code.